



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Activity Diagrams

Francesco Guerra
francesco.guerra@unimore.it

Activity diagrams

- ▶ Use cases show *what* your system should do.
- ▶ Activity diagrams allow you to specify *how* your system will accomplish its goals.
 - ▶ high-level actions chained together to represent a process occurring in your system.
- ▶ Activity diagrams are particularly good at modeling *business processes*.
 - ▶ A business process is a set of coordinated tasks that achieve a business goal.
- ▶ Some business process management (BPM) tools allow you to define business processes using activity diagrams and then execute them.
 - ▶ This allows you to define and execute, for example, a payment approval process where one of the steps invokes a credit card approval
- ▶ Activity diagrams use symbols similar to the widely-known flowchart notation; therefore, they are useful for describing processes to a broad audience.
 - ▶ They have their roots in flowcharts, as well as UML state diagrams, data flow diagrams, and Petri Nets.

Activity diagrams and Use Cases

Use case name: Create a new Blog Account

Goal In Context: A new or existing author requests a new blog account from the Administrator.

Preconditions: The system is limited to recognized authors, and so the author needs to have appropriate proof of identity.

Successful End Condition: A new blog account is created for the author.

Failed End Condition: The application for a new blog account is rejected.

Primary Actors: Administrator.

Secondary Actors: Author Credentials Database.

Trigger: The Administrator asks the Content Management System to create a new blog account.

Activity diagrams and Use Cases (2)

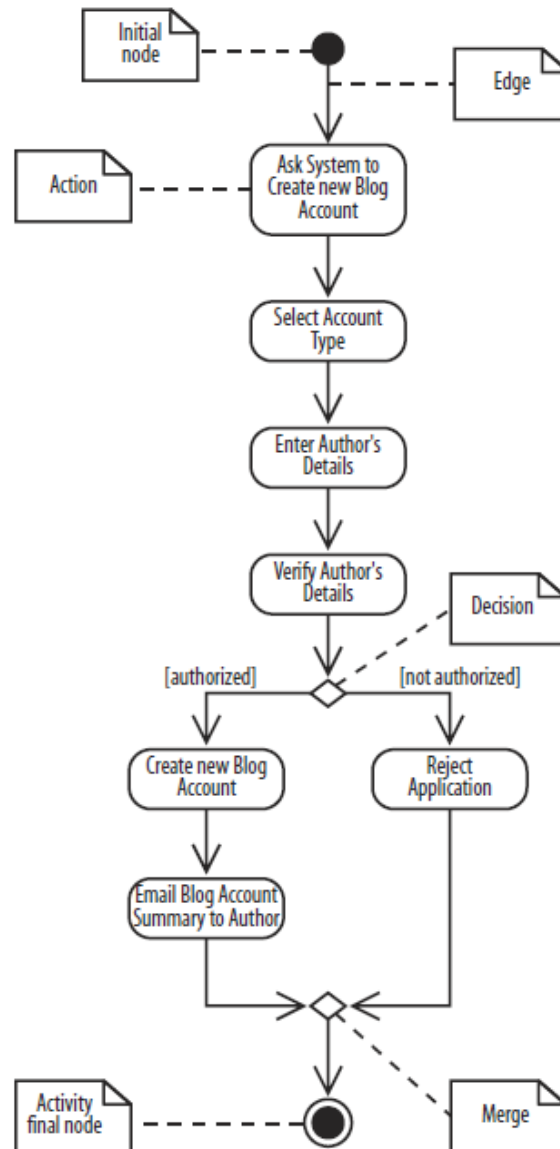
Main Flow Step Action

- 1 The Administrator asks the system to create a new blog account.
- 2 The Administrator selects an account type.
- 3 The Administrator enters the author's details.
- 4 The author's details are verified using the Author Credentials Database.
- 5 The new blog account is created.
- 6 A summary of the new blog account's details are emailed to the author.

Extensions Step Branching Action

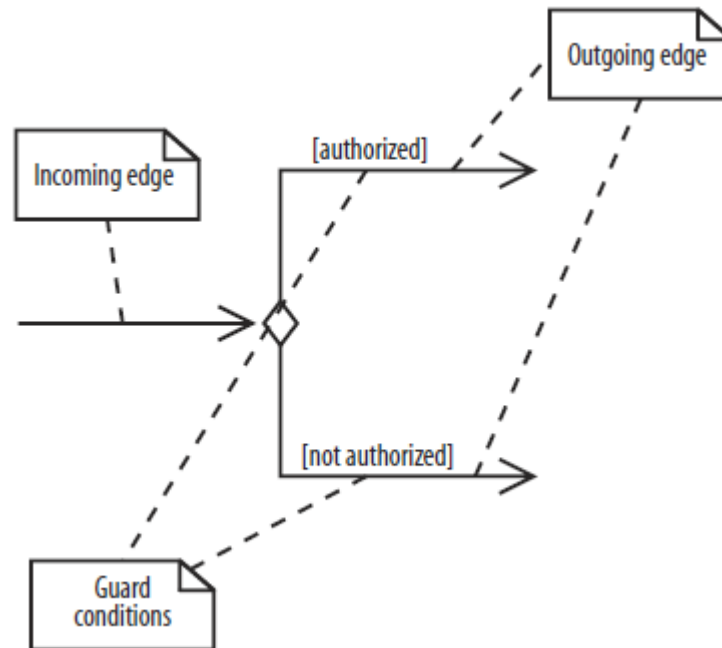
- 4.1 The Author Credentials Database does not verify the author's details.
- 4.2 The author's new blog account application is rejected.

Activity diagrams and Use Cases (3)



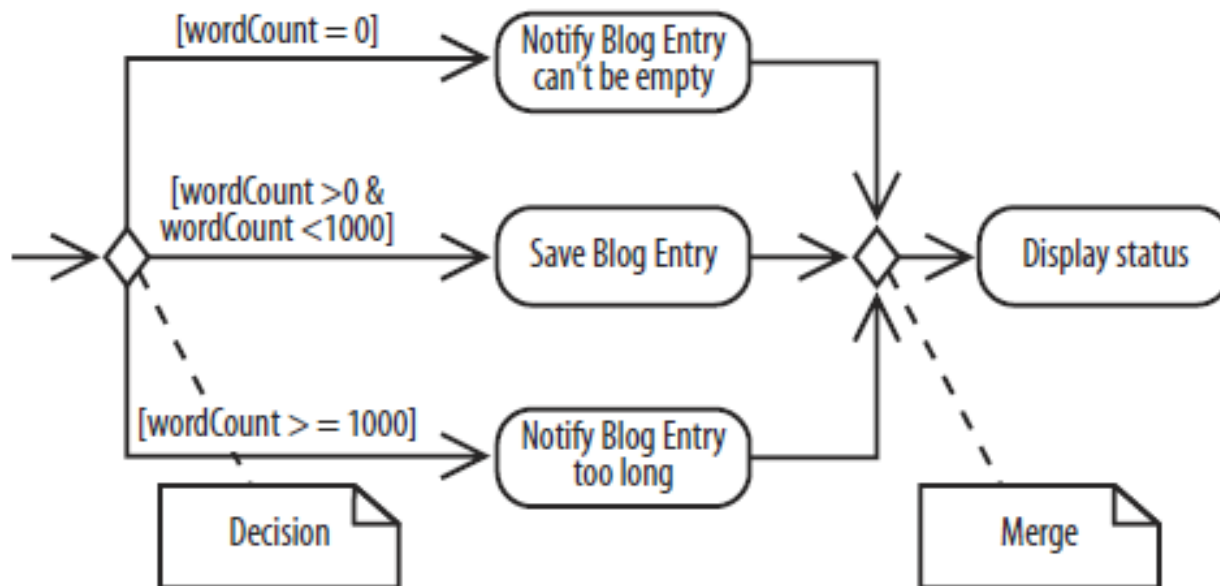
Decisions and Merges

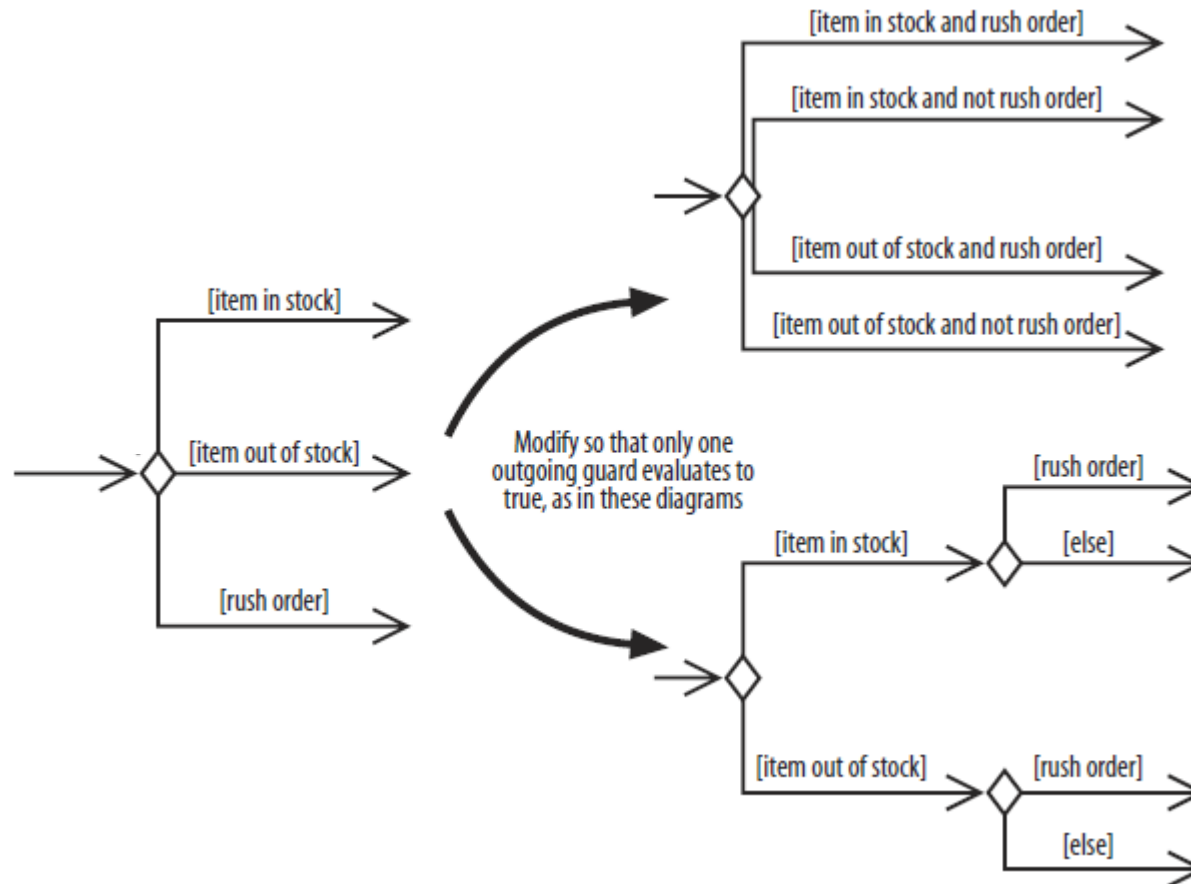
- ▶ *Decisions* are used when you want to execute a different sequence of actions depending on a condition.
- ▶ Each branched edge contains a *guard condition* written in brackets determining which edge is taken after a decision node.



Decisions and Merges (2)

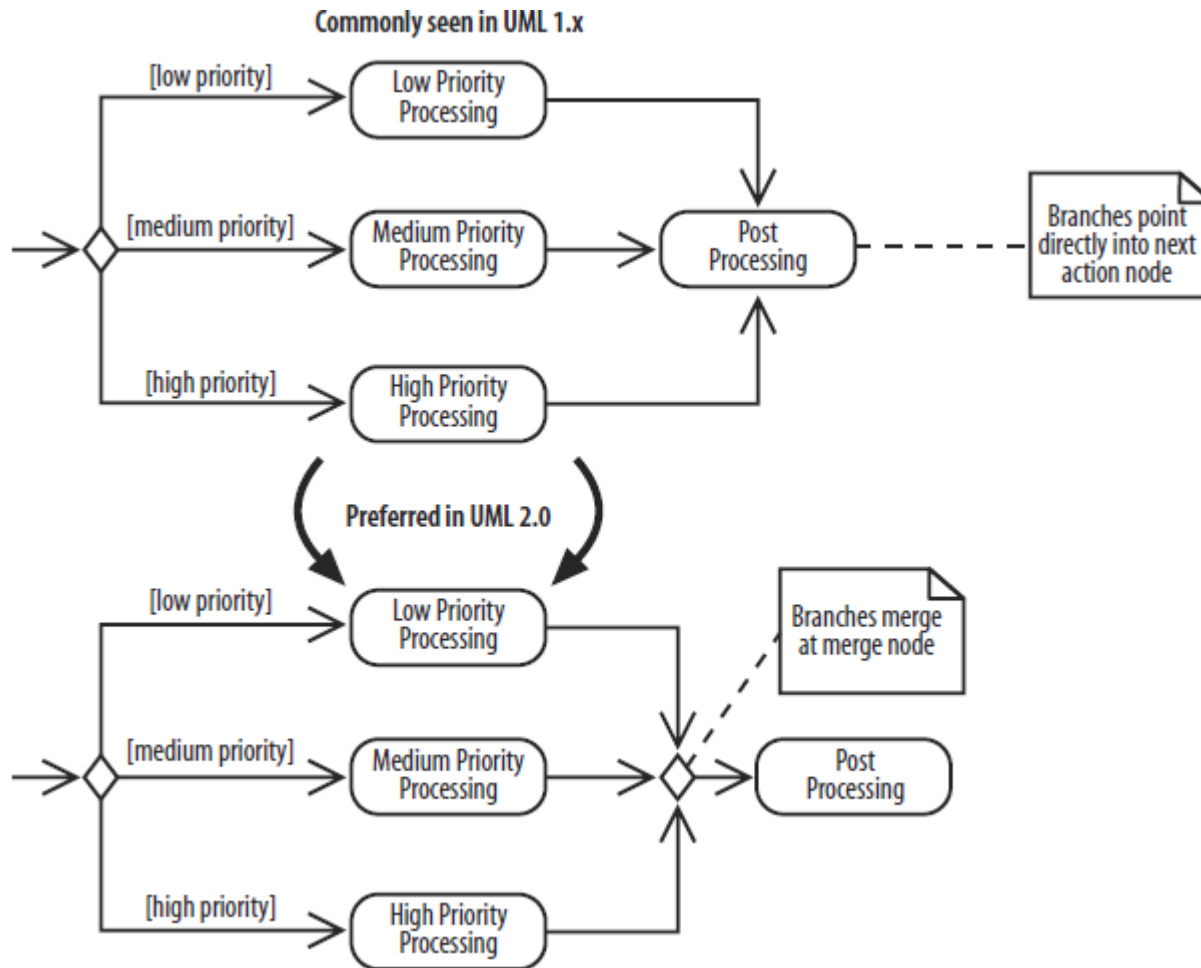
- ▶ The branched flows join together at a *merge* node, which marks the end of the conditional behavior started at the decision node



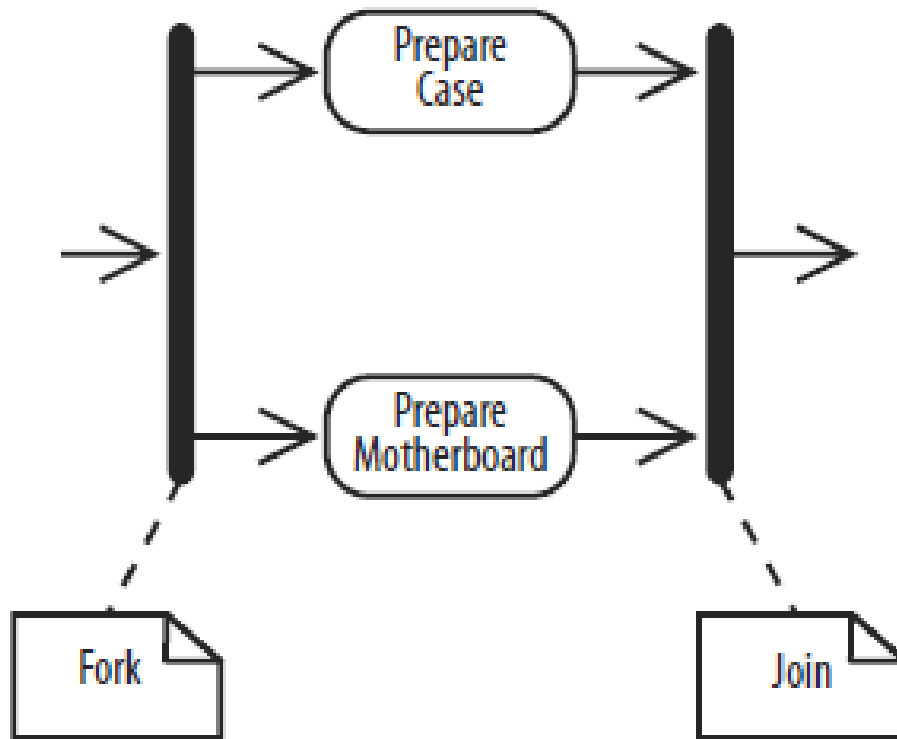


Another issue is if you don't have a complete guard -- > the activity is frozen at the decision node

Doing Multiple Tasks at the Same Time



forks and joins



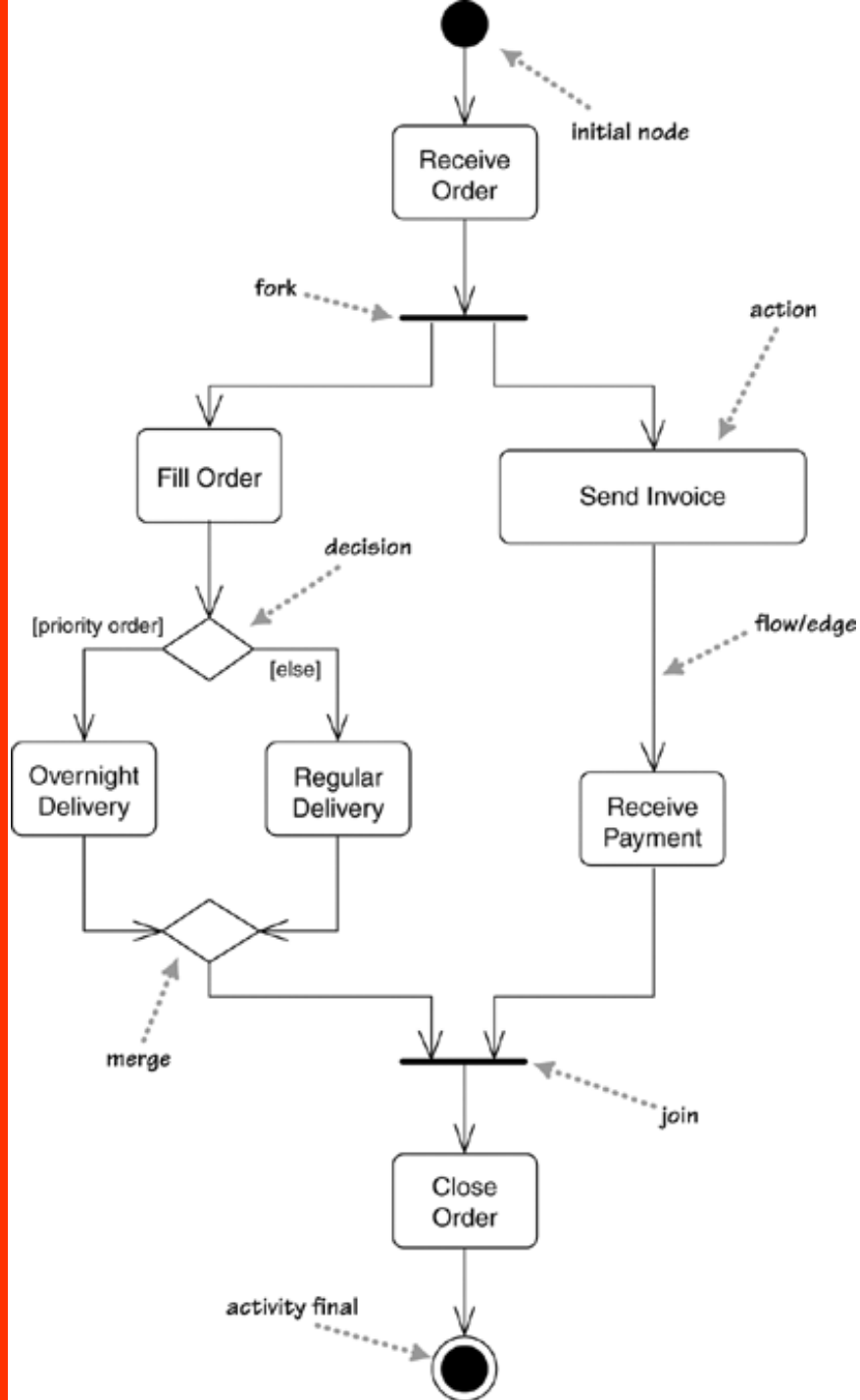
When actions occur in parallel, it doesn't necessarily mean they will finish at the same time.

One task will most likely finish before the other.

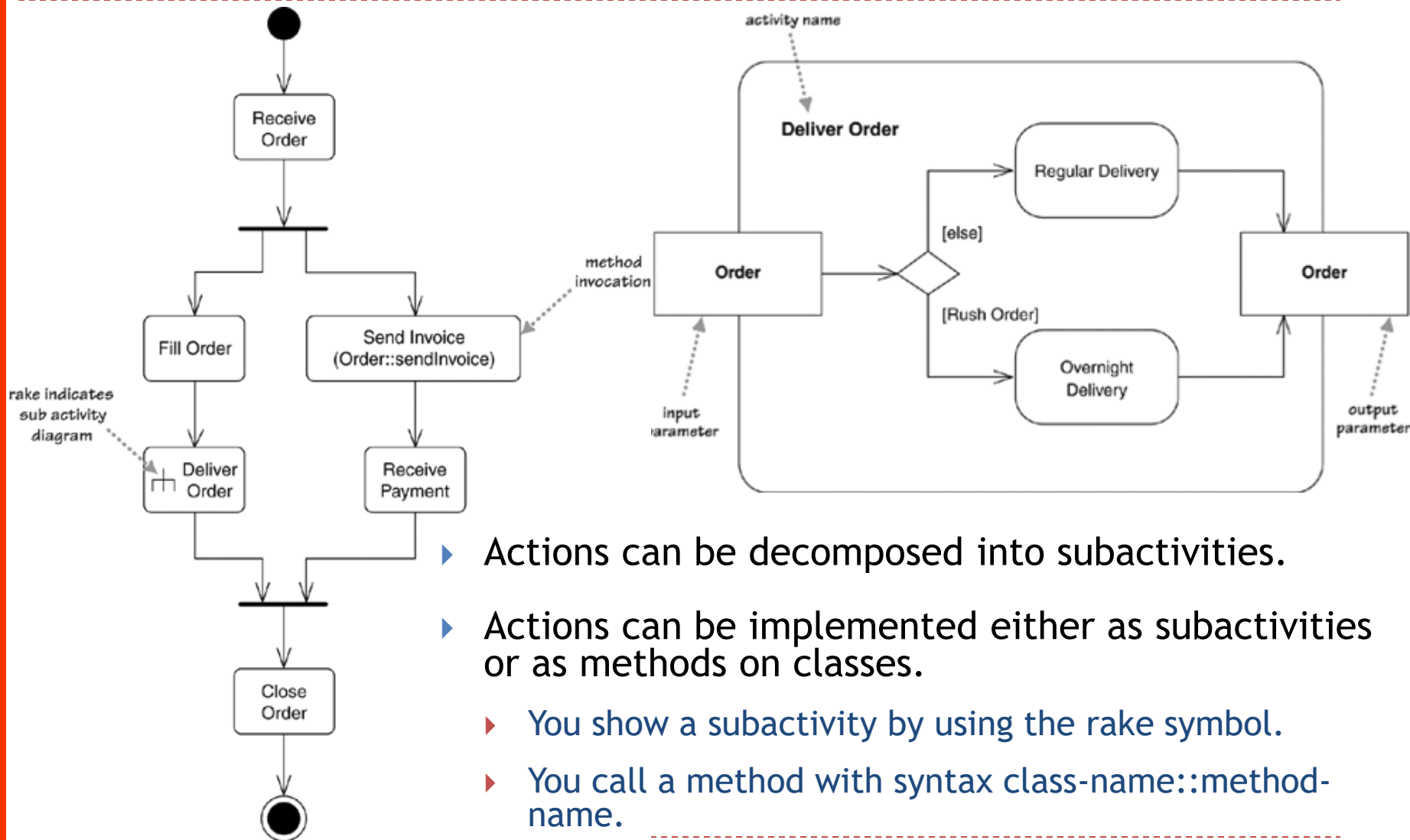
However, the join prevents the flow from continuing past the join until all incoming flows are complete.

Activity diagrams

- Example



Decomposing an Action

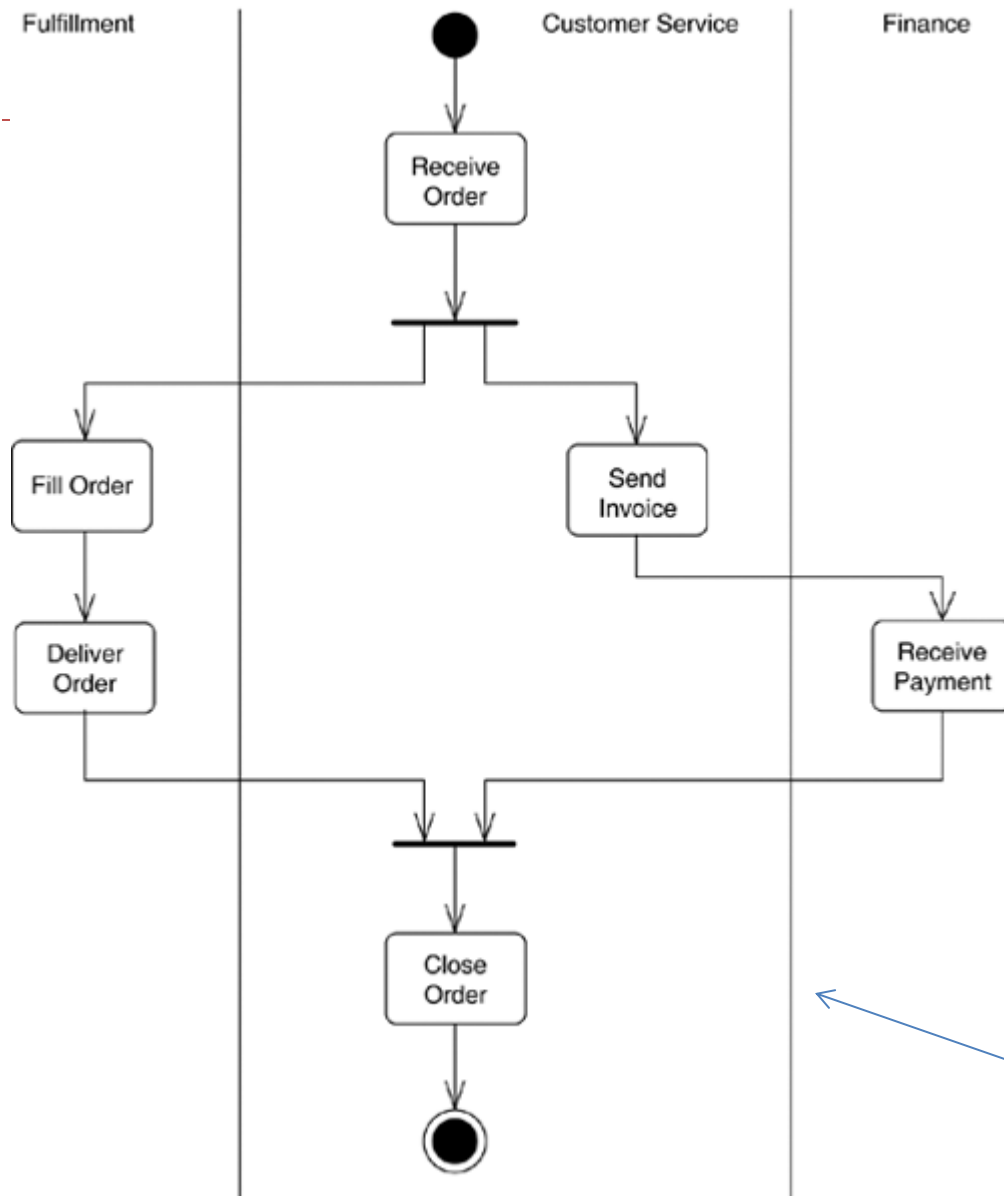


- ▶ Actions can be decomposed into subactivities.
- ▶ Actions can be implemented either as subactivities or as methods on classes.
 - ▶ You show a subactivity by using the rake symbol.
 - ▶ You call a method with syntax `class-name::method-name`.
 - ▶ You can write a code fragment into the action symbol.

Partitions

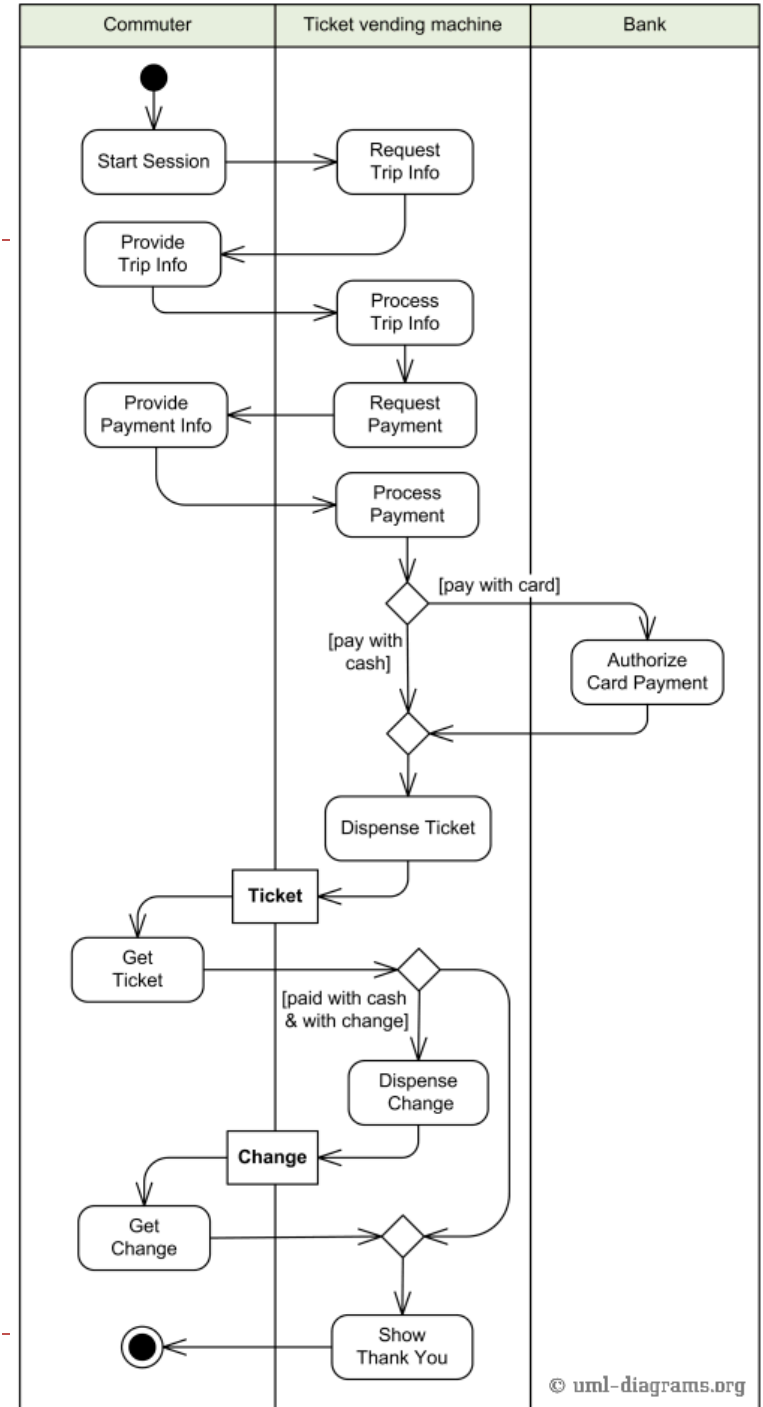
- ▶ Activity diagrams tell you what happens, but they do not tell you who does what.
 - ▶ In programming, this means that the diagram does not convey which class is responsible for each action.
 - ▶ In business process modeling, this does not convey which part of an organization carries out which action.
- ▶ This isn't necessarily a problem; often, it makes sense to concentrate on what gets done rather than on who does what parts of the behavior.
- ▶ If you want to show who does what, you can divide an activity diagram into partitions, which show which actions one class or organization unit carries out.

Partitions



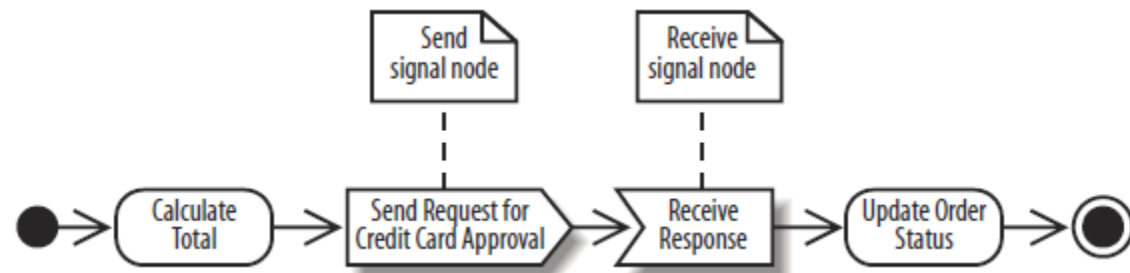
swim lanes

Example



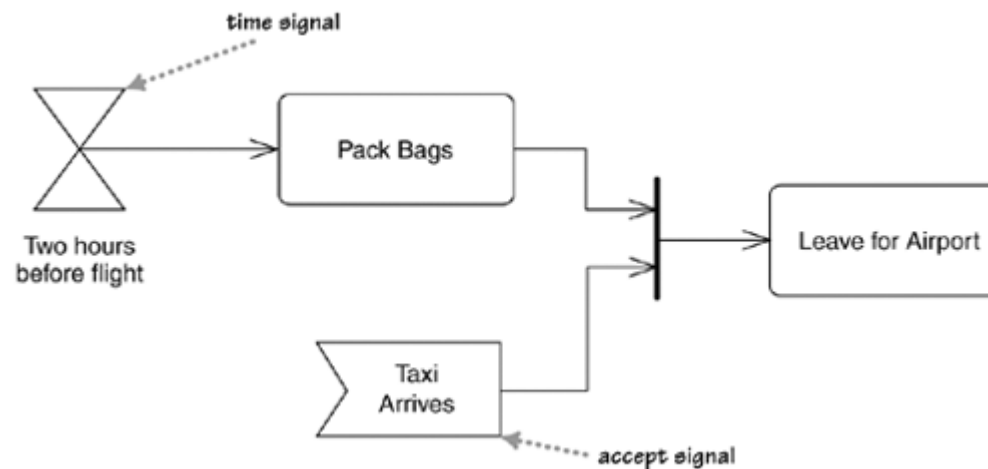
Sending and Receiving Signals

- ▶ In activity diagrams, *signals* represent interactions with external participants.
- ▶ Signals are messages that can be sent or received, as in the following examples:
 - ▶ Your software sends a request to the credit card company to approve a credit card transaction, and your software receives a response from the credit card company (sent and received, from the perspective of your credit card approval activity).
- ▶ A *receive signal* has the effect of waking up an action in your activity diagram.
- ▶ The recipient of the signal knows how to react to the signal and expects that a signal will arrive at some time but doesn't know exactly when.
- ▶ *Send signals* are signals sent to an external participant.
 - ▶ When that external person or system receives the message, it probably does something in response, but that isn't modeled in your activity diagram.

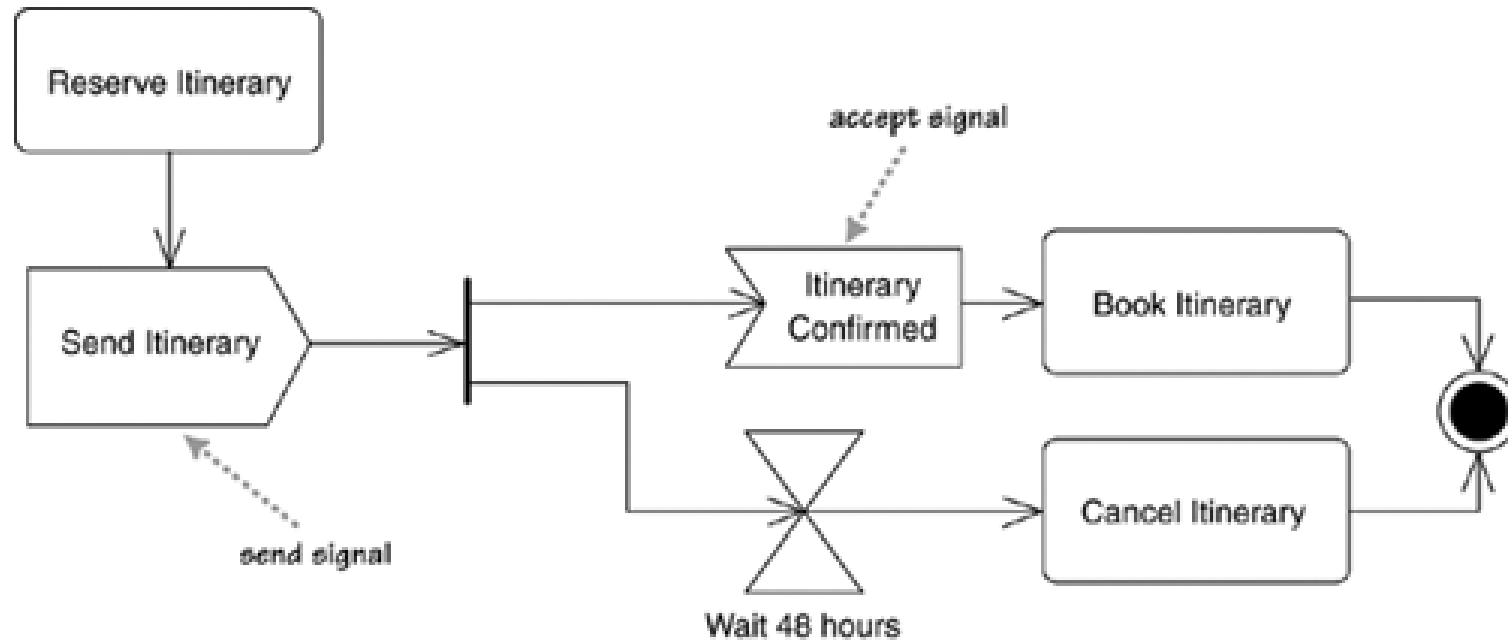


Signals

- ▶ Actions can also respond to signals.
 - ▶ A time signal occurs because of the passage of time.
 - ▶ Such signals might indicate the end of a month in a financial period or each microsecond in a real-time controller.



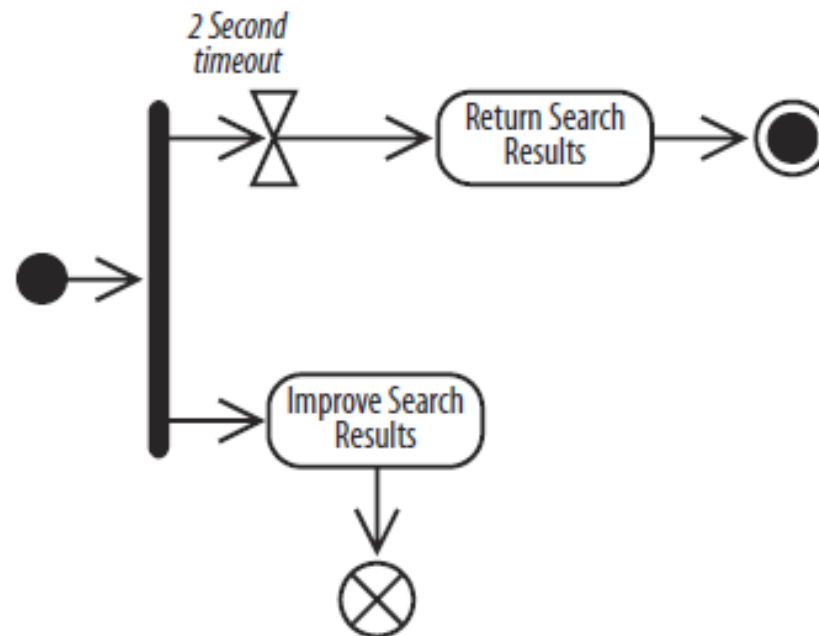
Signals (2)



the node is *always* waiting for a signal

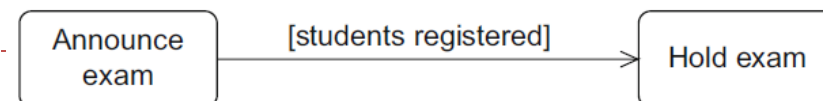
Ending a Flow

- ▶ A *flow final* node terminates its own path—not the whole activity.
 - ▶ This is a new feature of UML 2.0



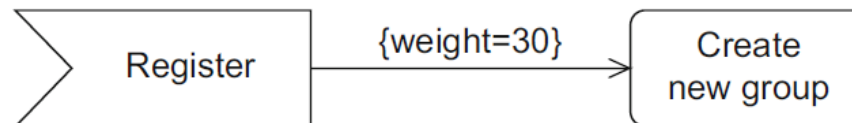
Control Flows

- ▶ To integrate the dynamic behavior aspects into the diagram, we need execution semantics, meaning that we have to specify exactly how an activity diagram is executed.
- ▶ The token concept, as introduced in Petri nets, is the basis for the execution semantics of the activity diagram.
 - ▶ A token is a virtual coordination mechanism that describes the execution exactly.
 - ▶ Virtual means that the tokens are not physical components of the diagram.
 - ▶ If an action receives a token, the action is active and can be executed.
 - ▶ Once the action has ended, it passes the token to a subsequent node via an edge and thus triggers the execution of this action.
 - ▶ The passing of a token can be prevented by a *guard* evaluating to false.
 - A guard is specified in square brackets.



Control Flows

- ▶ When an action is executed, usually one token of each of its incoming edges is consumed.
- ▶ Alternatively, a *weight* may be placed on an edge to allow a certain number of tokens to be consumed at that edge by a single execution.
 - ▶ The weight of an edge is specified in curly brackets with the keyword *weight*.
 - ▶ It is always a whole number greater than or equal to zero.
 - ▶ If the weight is zero, all or *, this means that all tokens present are consumed.
 - ▶ If no weight is specified, 1 is assumed as the default value.



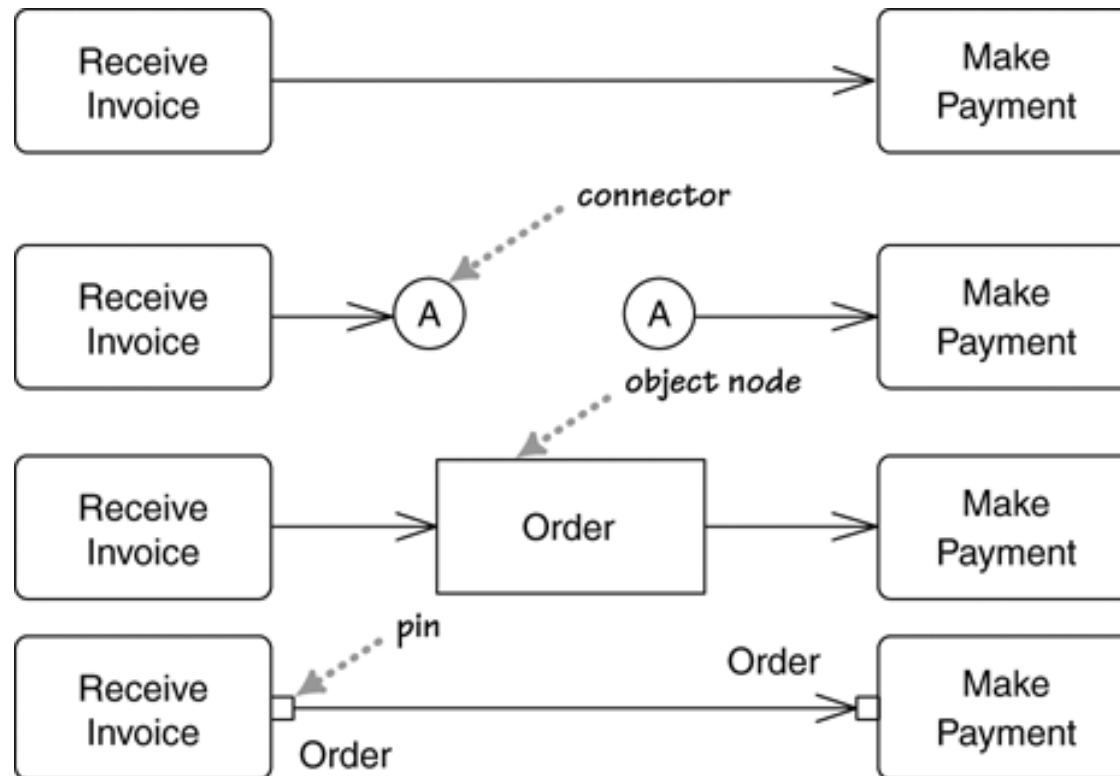
Tokens

- ▶ The initial node creates a token, which then passes to the next action, which executes and then passes the token to the next.
- ▶ At a fork, one token comes in, and the fork produces a token on each of its outward flows.
- ▶ Conversely, on a join, as each inbound token arrives, nothing happens until all the tokens appear at the join; then a token is produced on the outward flow.
- ▶ You can visualize the tokens with coins or counters moving across the diagram.
 - ▶ As you get to more complicated examples of activity diagrams, tokens often make it easier to visualize things.

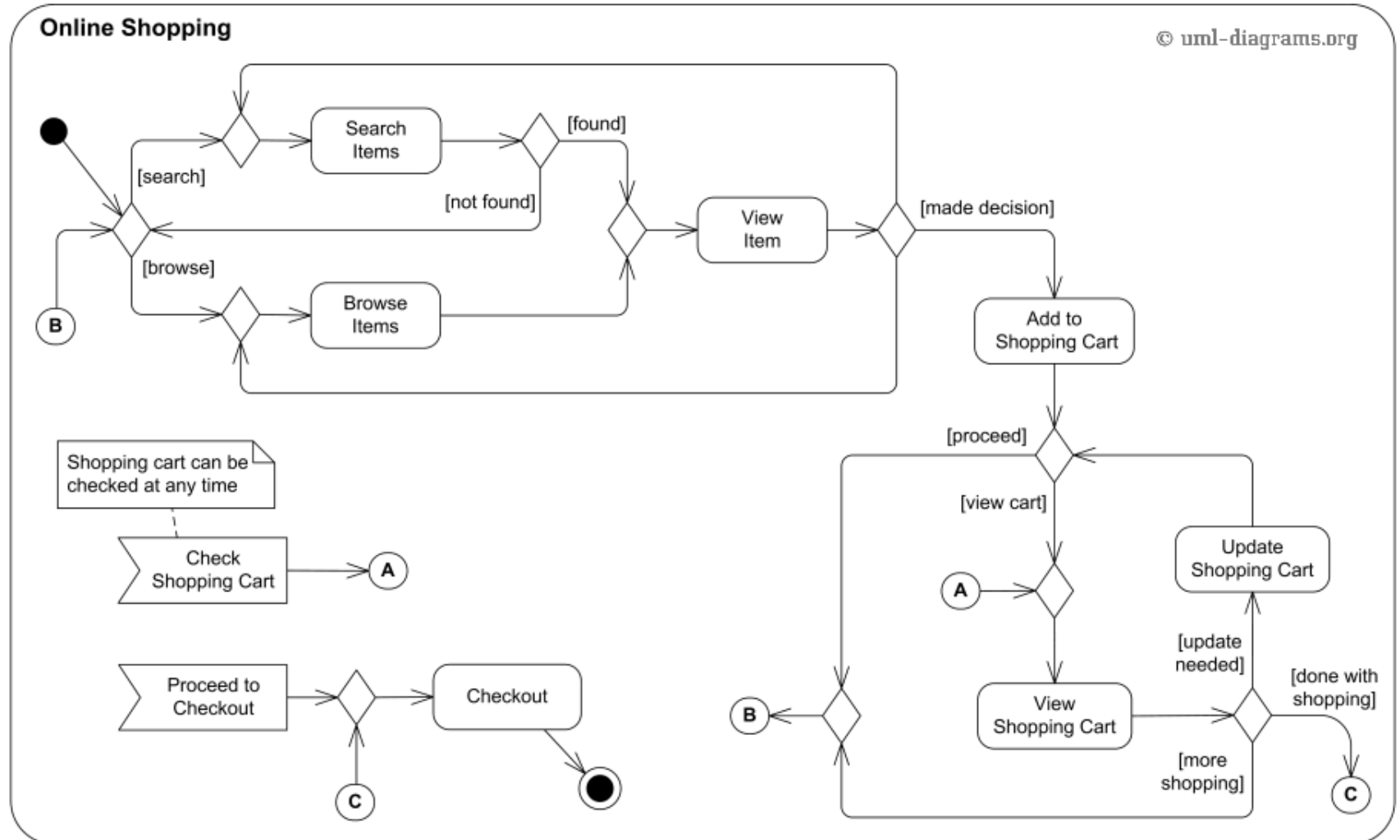
Flows and Edges

- ▶ UML 2 uses the terms flow and edge synonymously to describe the connections between two actions.
 - ▶ The simplest kind of edge is the simple arrow between two actions.
 - ▶ You can give an edge a name if you like.
- ▶ If you're having difficulty routing lines, you can use connectors, which simply save you having to draw a line the whole distance.
 - ▶ When you use connectors, you must use them in pairs: one
- ▶ The simplest edges pass a token that has no meaning other than to control the flow.
- ▶ However, you can also pass objects along edges; the objects then play the role of tokens, as well as carry data.
 - ▶ If you are passing an object along the edge, you can show that by putting a class box on the edge, or you can use pins on the actions.

Flows and Edges (2)

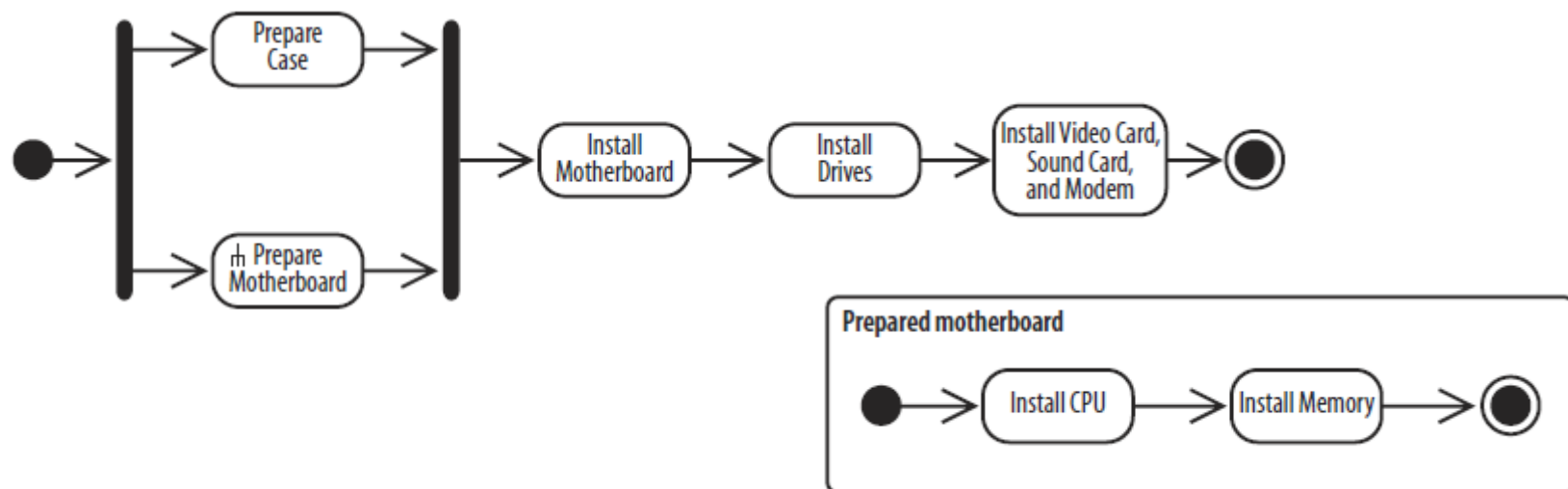


Example



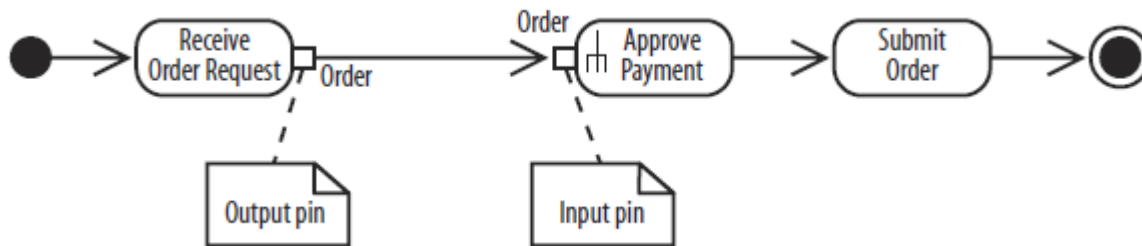
Calling Other Activities - example

- ▶ As detail is added to your activity diagram, the diagram may become too big, or the same sequence of actions may occur more than once.
- ▶ When this happens, you can improve readability by providing details of an action in a separate diagram, allowing the higher level diagram to remain less cluttered

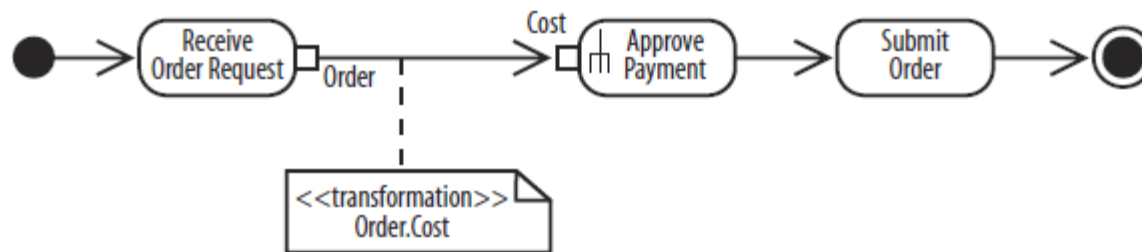


Pins and Transformations

- ▶ Actions can have parameters.
 - ▶ You don't need to show information about parameters, but if you wish, you can show them with pins.
 - ▶ An *input pin* means that the specified object is input to an action.
 - ▶ An *output pin* means that the specified object is output from an action.
- ▶ When you're drawing an activity diagram strictly, you have to ensure that the output parameters of an outbound action match the input parameters of another.
 - ▶ If they don't match, you can indicate a transformation to get from one to another.
 - ▶ The transformation must be an expression that's free of side effects: essentially, a query on the output pin that supplies an object of the right type for the input pin.



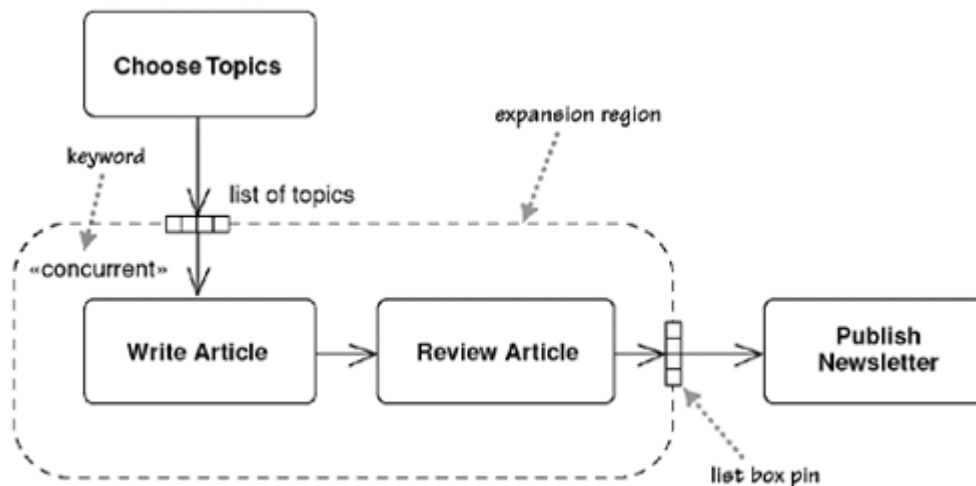
Pins and Transformations (2)



- ▶ Approve Payment action requires the Cost object as input and shows how this data is obtained from the Order object using the transformation specified in a note.

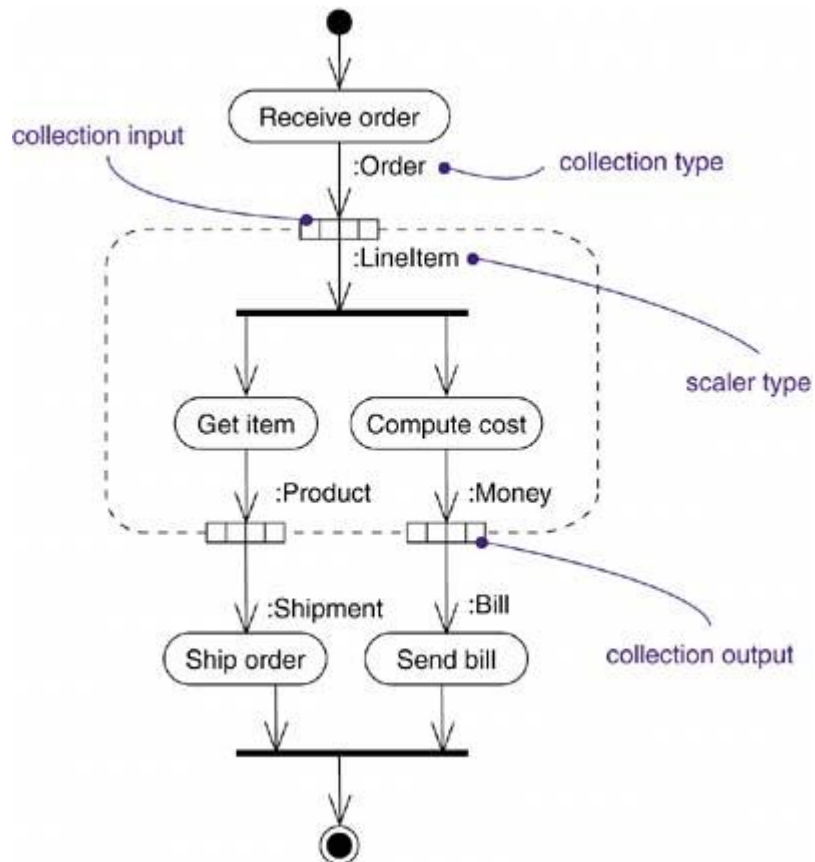
Expansion Regions

- ▶ Useful in situations in which one action's output triggers multiple invocations of another action.
- ▶ An expansion region marks an activity diagram area where actions occur once for each item in a collection.

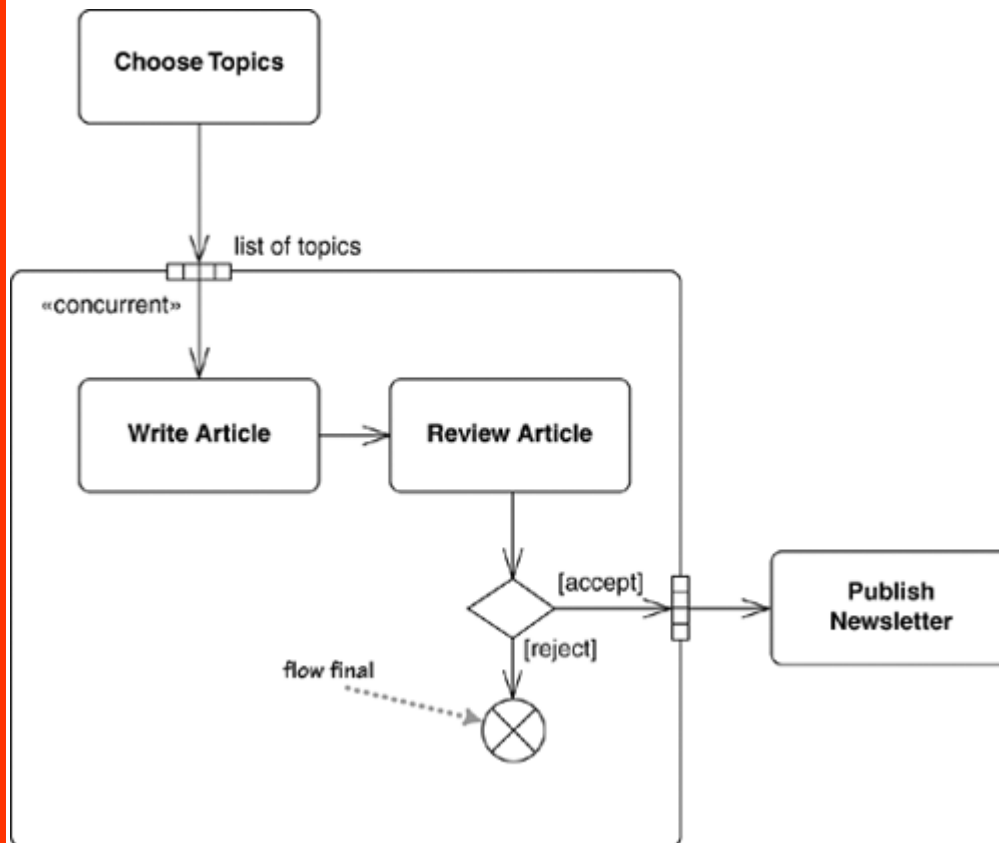


Iterative expansion regions

- Iterative regions must fully process each input element one at a time.



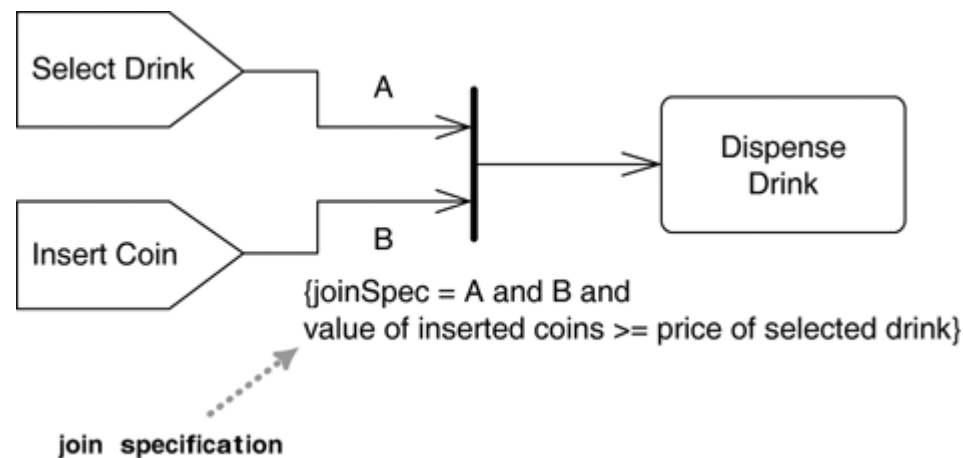
Flow Final in an expansion region



- ▶ Once you get multiple tokens, as in an expansion region, you often get flows that stop even when the activity as a whole doesn't end.
- ▶ A flow final has to be inserted

Join Specifications

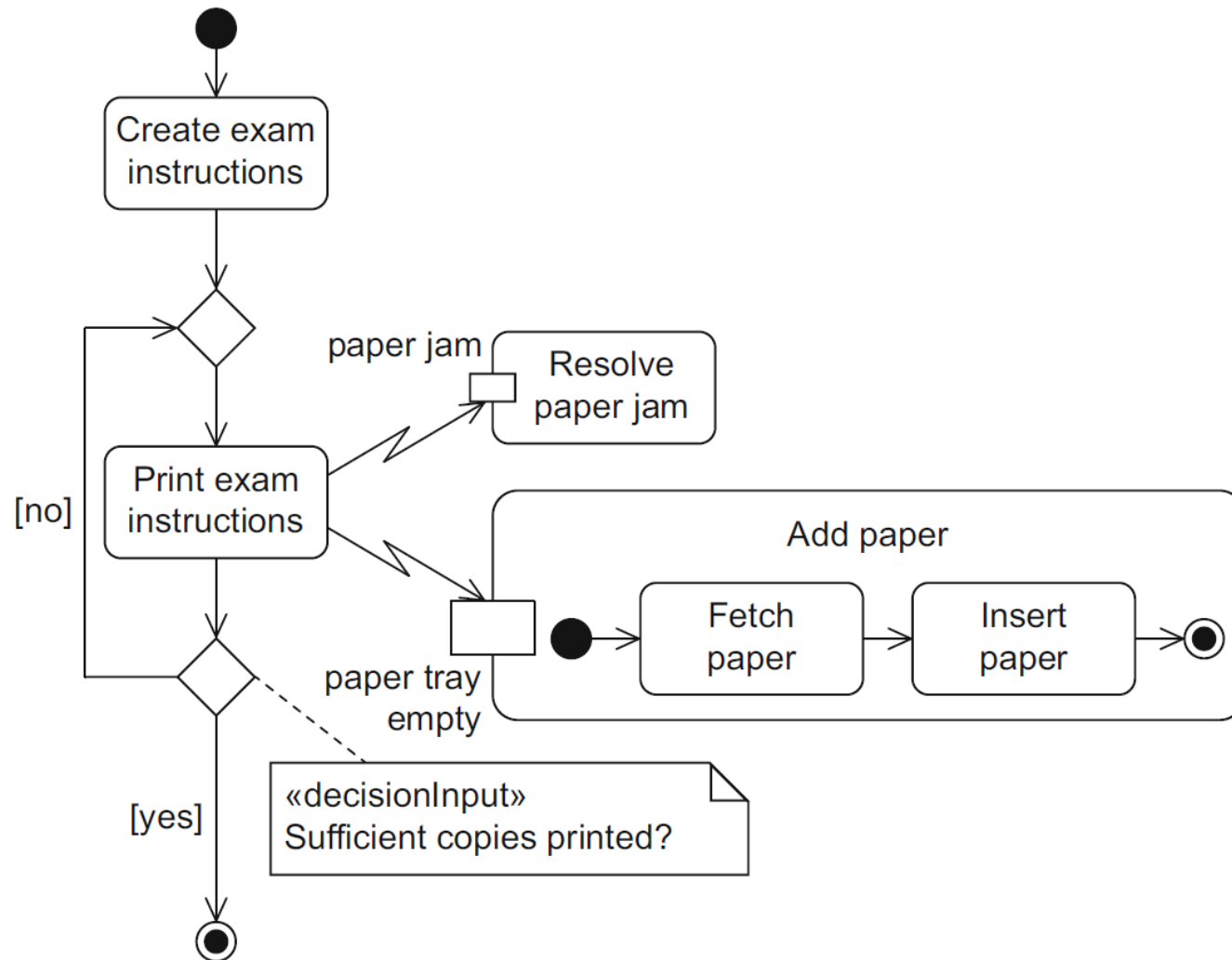
- ▶ In some cases, particularly when you have a flow with multiple tokens, it's useful to have a more involved rule.
- ▶ A join specification is a Boolean expression attached to a join.
- ▶ Each time a token arrives at the join, the join specification is evaluated and if true, an output token is emitted



Exception Handling

- ▶ If an error occurs during an action, the execution is terminated.
 - ▶ There is no guarantee that the action will deliver the expected output.
- ▶ An *exception handler* is activated when an exception occurs.
 - ▶ Exception handlers allows you to define how the system reacts in a specific error situation.
 - ▶ You specify an exception handler for a specific type of error
 - ▶ You can use different exception handlers for different errors.
 - ▶ If an error situation occurs, all tokens in the action are deleted.
- ▶ If there is a matching exception handler:
 - ▶ this replaces the content of the action concerned and instead,
 - ▶ the content of the exception handler is executed.
- ▶ The sequence then continues as the regular path of the activity as if the defective action had ended normally.

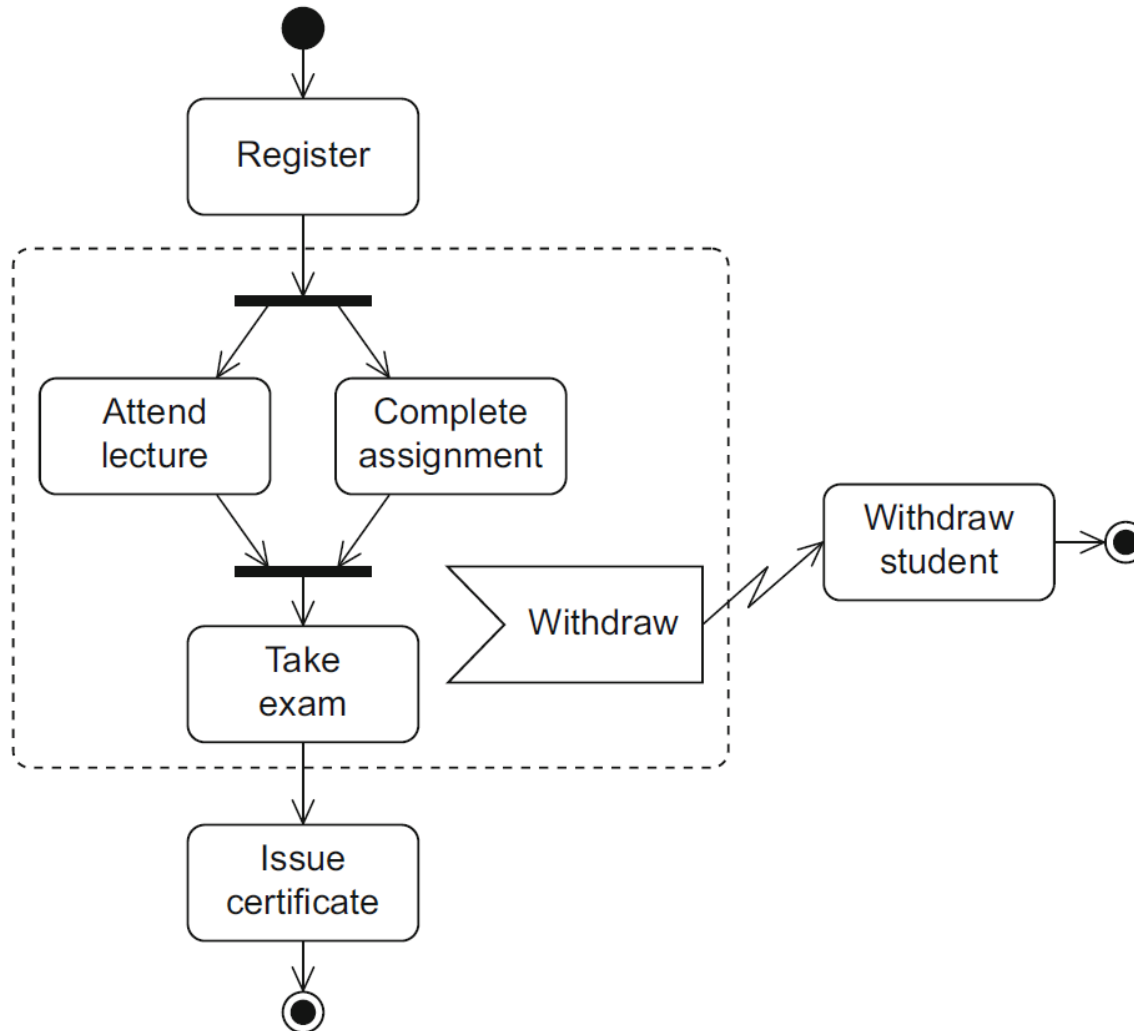
Example



Interruptible activity region

- ▶ The *interruptible activity region* offers a way to handle exceptions.
 - ▶ You can define a group of actions whose execution is to be terminated immediately if a specific event occurs.
 - ▶ The interruptible activity region is depicted as a dashed rectangle with rounded corners that encloses the relevant actions.
 - ▶ The execution of these events is monitored for the occurrence of a specific event, for example an error.
- ▶ If the event does occur during this execution, then as a certain behavior is executed.
 - ▶ If the modeled event occurs, all control tokens in the interruptible activity region are deleted
 - ▶ The action that the accept event action points to is activated.

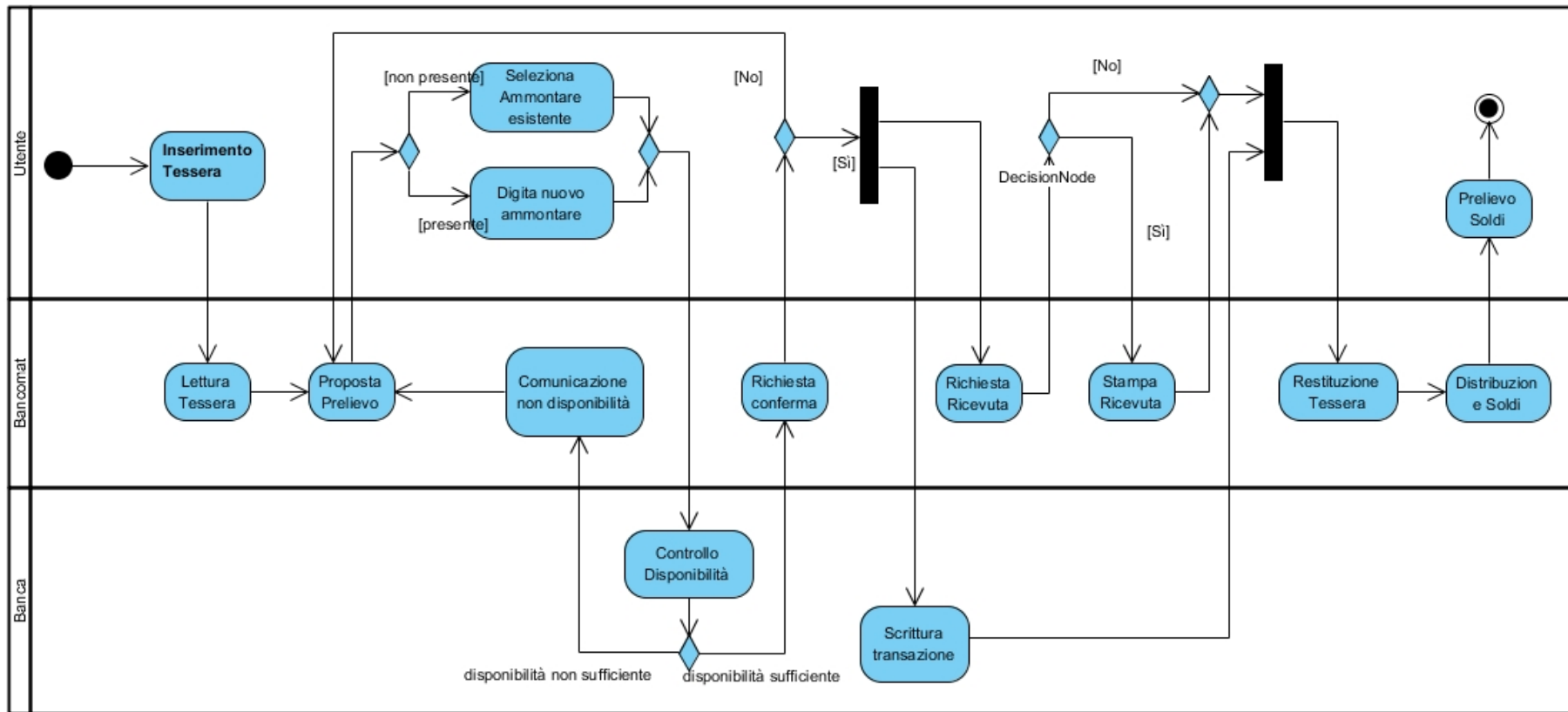
Example



Esercizio

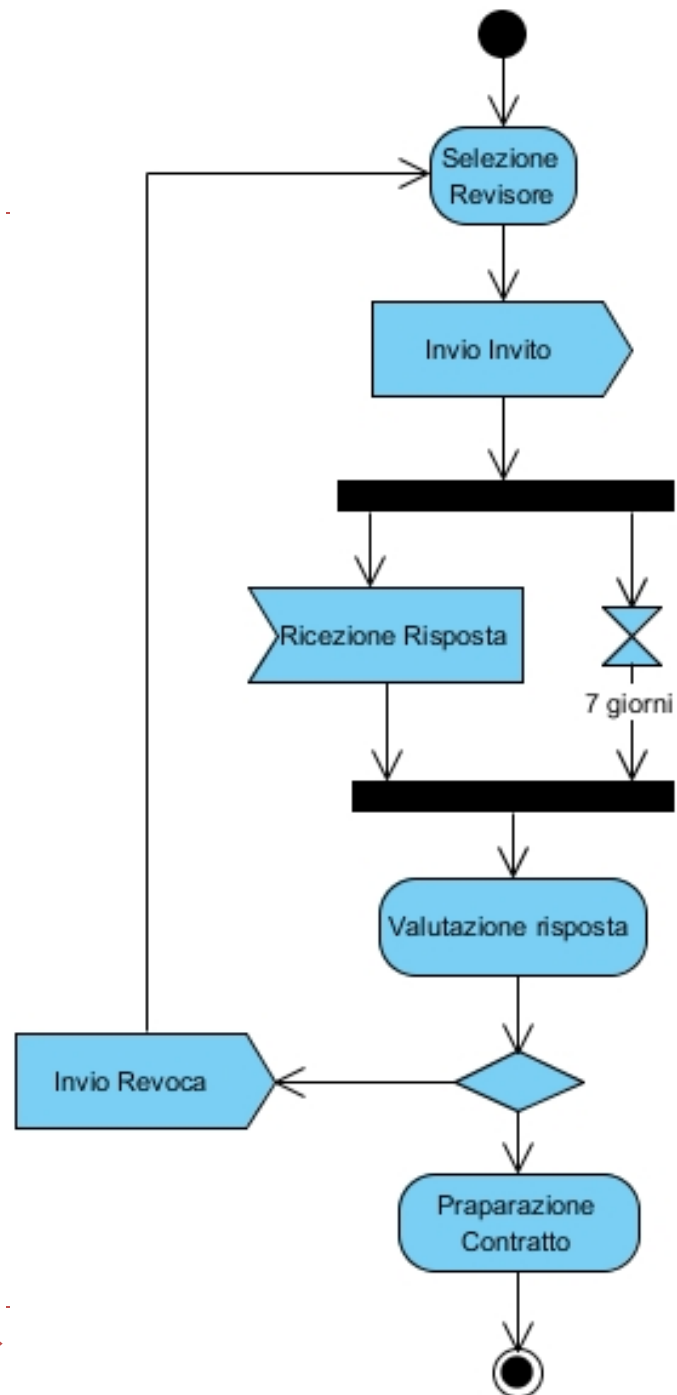
- ▶ Si modellino tutti i passi necessari per la richiesta di un prelievo a uno sportello di un bancomat. Per rendere più semplice la visualizzazione del diagramma, si considerino le attività svolte dall'utente, dalla banca e dallo sportello attraverso 3 partizioni distinte

Una possibile soluzione



Esercizio

- ▶ Un noto editore internazionale ha bisogno di un revisore per un libro scientifico. L'editore, dopo avere identificato un possibile candidato, invia un messaggio di invito. A questo punto le possibilità sono due: la persona risponde accettando o declinando l'invito, oppure l'editore non ottiene risposte.
- ▶ A fronte dell'accettazione, si passa alla stesura del contratto. Altrimenti (in caso di rifiuto o di non risposta dopo 7 giorni), l'editore manda un messaggio di ringraziamento e passa all'individuazione di un nuovo candidato.



join spec =Ricezione risposta OR 7 giorni

Una possibile
soluzione

Esercizio

- ▶ Si modelli un particolare iter per sostenere un esame all'università.
- ▶ Lo studente deve decidere quale esame deve sostenere. Quindi deve studiare e iscriversi. L'iscrizione avviene attraverso SMS mandato al sistema di gestione dell'Università e a cui viene mandata una risposta con il giorno e l'aula. La risposta può non essere immediata e richiedere il controllo di vincoli amministrativi.
- ▶ Dopo avere sostenuto l'esame, lo studente deve aspettare l'esito dell'esame. Se negativo, l'intero processo deve essere eseguito nuovamente.



References

- ▶ Martin Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language , Pearson Addison Wesley
- ▶ Russ Miles, Kim Hamilton: Learning UML 2.0 A Pragmatic Introduction to UML, O'Reilly Media, 2006
- ▶ Martina Seidl, Marion Scholz, Christian Huemer, Gerti Kappel: UML @ Classroom - An Introduction to Object-Oriented Modeling, Springer 2015

MATERIALE DIDATTICO INTERNO DEL CORSO DI Ingegneria del Software.

La diffusione e pubblicazione on line del materiale è assolutamente proibita.